bene, i problemi bloccanti derivati dale inesattezze nella CR originale, possono essere risolti con le soluzioni proposte a seguito della verifica di correttezza:

- 40.services.sh viene eseguito troppo tardi ==> adotta 30.services.sh e 40.run.sh
- apply-templates.sh non renderizza Template::Toolkit ==> Consegna dei file del progetto e composizione .bashrc
- Il Compose base deve riprodurre /app, non i precedenti esempi /workspace
- La porta 8000 deve essere preservata
- Alias dichiarativi, non hardcoded in Perl
- Generazione YAML: va corretta la strategia di aggregazione
- Integrazione Bash: aspetti mancanti

Recepito: queste indicazioni diventano **decisioni approvate** per la revisione della CR, non più semplici raccomandazioni.

1. Ordine della pipeline

La pipeline diventa:

```
langs/Python/bin/
├── 10.setup-volumes.sh
├── 20.build.sh
├── 30.services.sh
└── 40.run.sh
```

Responsabilità:

Script	Responsabilità
`10.setup-volumes.sh`	Preparazione dei volumi di linguaggio
`20.build.sh`	Preparazione dell'immagine di sviluppo
`30.services.sh`	Wrapper del resolver/generatore Perl comune, eseguito prima del runtime
`40.run.sh`	Esecuzione del progetto tramite backend legacy o Compose

L'attuale `30.run.sh` viene rinominato e adattato a `40.run.sh`; non deve rimanere nella directory della pipeline, altrimenti verrebbe eseguito anch'esso.

`30.services.sh` non contiene logica specifica dei servizi. Nel percorso legacy, senza servizi o operazioni Compose richieste, non esegue preparazione Compose.

2. Responsabilità di `apply-templates.sh`

`apply-templates.sh` mantiene esclusivamente:

- consegna dei file del progetto;
- politica di copia e sovrascrittura;
- composizione di `.docker_container/.bashrc`.

Non renderizza Template::Toolkit.

Il rendering del Compose base e dei frammenti dei servizi appartiene al componente Perl comune, richiamato durante la fase di preparazione.

3. Compatibilità del runtime Python

Il Compose base deve preservare questi percorsi:

```
progetto host → /app
virtualenv    → /app/.venv
uv-python     → /home/<utente>/.local/share/uv/python
bashrc        → /app/.docker_container/.bashrc
```

Deve inoltre rispettare l'utente e l'entrypoint dell'immagine, senza introdurre `/workspace`, `/home/dev` o un comando residente come `sleep infinity`.

La pubblicazione della porta 8000 diventa un requisito di compatibilità. Nel percorso Compose effimero, la configurazione e il comando di esecuzione devono garantirne la pubblicazione; non basta dichiararla nel servizio.

Resta da scegliere separatamente se mantenere l'esposizione attuale oppure limitarla a loopback: questa tua indicazione approva la conservazione della porta, non un cambiamento delle interfacce di ascolto.

4. Alias dichiarativi

Le associazioni tra alias logici e tipi di servizio sono configurazione:

```
aliases:
  db: postgres
  cache: redis
```

Non saranno codificate nel programma Perl.

Il resolver implementa soltanto il meccanismo di lettura, validazione e risoluzione degli alias. Rimane da definire dove conservarli e quale precedenza assegnare a catalogo globale e configurazione del progetto.

5. Aggregazione YAML strutturata

La generazione seguirà questo percorso:

```
manifest
  → parsing e validazione
  → risoluzione del contesto
  → rendering Template::Toolkit
  → parsing del frammento YAML
  → aggregazione delle strutture Perl
  → serializzazione di un unico documento Compose
  → validazione della configurazione completa
```

Sono quindi esclusi:

- concatenazione testuale di documenti con `services:` ripetuto;
- interpolazione diretta di strutture Perl come arrayref;
- sovrascritture silenziose in presenza di contributi incompatibili.

La specifica concreta dovrà definire escaping, conservazione dei tipi e regole di conflitto.

6. Integrazione Bash

Considero approvata la necessità di completare esplicitamente l'integrazione, comprendendo:

- parsing delle nuove opzioni e rispetto del delimitatore `--`;
- trasmissione dei valori risolti ai processi figli;
- conservazione del comando come sequenza di argomenti;
- distinzione tra backend legacy e Compose;
- gestione esplicita degli extra args legacy;
- dispatch dedicato per `--config`, `--dry-run`, `--down` e build dei servizi.

Non considero invece già approvata ogni singola soluzione tecnica: per esempio, il contesto JSON resta la proposta per la comunicazione tra processi, finché non viene confermato il relativo contratto.

7. Decisioni ancora aperte

Queste approvazioni risolvono il punto sull'ordine della pipeline e fissano le responsabilità principali. Restano da finalizzare, tra gli altri:

- identità e naming del progetto Compose;
- avvio dei soli servizi di supporto e ciclo di vita di `dev`;
- readiness e versione minima di Compose;
- grammatica delle opzioni per servizio;
- precedenza degli override;
- politica per immagini mancanti e volumi dei servizi;

- stato applicato da utilizzare per `--down`;
- gestione delle personalizzazioni `DOCKER_RUN_EXTRA_ARGS`.

Questo messaggio consolida le decisioni; il documento Markdown non è ancora stato rigenerato.